

This implementation plan provides a step-by-step guide for deploying the FRACA SERVCOM Inventory Management System to a production environment. It covers server requirements (PHP 8.2, MySQL, Nginx/Apache), environment configuration, database setup with seeded admin/manager/staff accounts, asset compilation via Vite, and web server configuration. Post-deployment tasks include caching optimizations, scheduled task configuration for low-stock alerts, security hardening, and verification procedures. A rollback plan and backup strategy are also included to ensure deployment reliability.

INVENTORY MANAGEMENT SYSTEM

IMPLEMENTATION PLAN DOC
PHILIP LEE 22/07341

IMPLEMENTATION PLAN (DEPLOYMENT PLAN)

FRACA SERVCOM Inventory Management System

Document Version: 1.0

Date: March 2026

Technology Stack: Laravel 11, PHP 8.2, MySQL, Bootstrap 5, Vanilla JavaScript (ES6+), Vite

1. Introduction

1.1 Purpose

This implementation plan outlines the steps required to deploy the FRACA SERVCOM Inventory Management System to a production environment. It covers server requirements, environment setup, database configuration, asset compilation, and post-deployment verification procedures.

1.2 Deployment Overview

The system is a Laravel 11 web application with session-based authentication and AJAX-driven UI. Deployment involves provisioning a web server, configuring environment variables, installing dependencies, running migrations and seeders, compiling frontend assets with Vite, and configuring the web server.

2. Server Requirements

2.1 Minimum Specifications

Component	Requirement
Web Server	Apache 2.4+ or Nginx 1.18+
PHP	8.2 or 8.3
PHP Extensions	BCMath, CType, JSON, Mbstring, OpenSSL, PDO, Tokenizer, XML, MySQLi, GD, cURL, Zip
Database	MySQL 8.0+ or MariaDB 10.6+
Web server	Apache 2.4+ or Nginx 1.18+
PHP	8.2 or 8.3
PHP Extensions	BCMath, CType, JSON, Mbstring, OpenSSL, PDO, Tokenizer, XML, MySQLi, GD, cURL, Zip
Database	MySQL 8.0+ or MariaDB 10.6+
RAM	2GB minimum (4GB recommended)
Storage	20GB minimum
OS	Ubuntu 22.04 LTS / 24.04 LTS (recommended)

2.2 Required Software

- Composer (PHP dependency manager)
- Node.js 18+ and NPM (for asset compilation)
- Git (for version control)

3. Pre-Deployment Checklist

- [] Server provisioned with PHP 8.2+ and MySQL
- [] Domain name pointed to server IP address
- [] SSL certificate installed (Let's Encrypt recommended)
- [] Git installed on server
- [] Composer installed globally
- [] Node.js 18+ and NPM installed
- [] Database created with appropriate credentials
- [] Firewall configured (ports 80, 443, 22 open)
- [] Backup strategy defined

4. Deployment Steps

4.1 Server Preparation (Ubuntu Example)

```
``bash
```

```
# Update system
```

```
sudo apt update && sudo apt upgrade -y
```

```
# Install PHP 8.2 and extensions
```

```
sudo apt install -y php8.2 php8.2-cli php8.2-common php8.2-mysql \  
php8.2-zip php8.2-gd php8.2-mbstring php8.2-curl php8.2-xml \  
php8.2-bcmath php8.2-tokenizer
```

```
# Install MySQL
```

```
sudo apt install -y mysql-server
```

```
# Install Composer
```

```
curl -sS https://getcomposer.org/installer | php  
sudo mv composer.phar /usr/local/bin/composer
```

```
# Install Node.js 18.x
```

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -  
sudo apt install -y nodejs
```

```
# Install Nginx (or Apache)
```

```
sudo apt install -y nginx
```

```
# Install Git
```

```
sudo apt install -y git
```

```
...
```

4.2 Application Deployment

```
``bash
```

```
# Clone repository
```

```
cd /var/www
```

```
git clone https://your-repository-url/fraca-inventory.git
```

```
cd fraca-inventory
```

```
#Install PHP dependencies
```

```
composer install --optimize-autoloader --no-dev
```

```
#Install Node dependencies
```

```
npm install
```

```
# Build frontend assets (production)
```

```
npm run build
```

```
# Copy environment file
cp .env.example .env

# Generate application key
php artisan key:generate

# Create storage link
php artisan storage:link
````
```

#### 4.3 Environment Configuration (.env)

```
```.env

APP_NAME="FRACA SERVCOM Inventory"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://yourdomain.com

# Database Configuration
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=fracas_inventory
DB_USERNAME=inventory_user
```

```
DB_PASSWORD=secure_password_here
```

```
# Session Configuration
```

```
SESSION_DOMAIN=yourdomain.com
```

```
SESSION_SECURE_COOKIE=true
```

```
# Cache Configuration
```

```
CACHE_DRIVER=redis # or file if redis not available
```

```
QUEUE_CONNECTION=database
```

```
...
```

4.4 Database Setup

```
```bash
```

```
Create database and user
```

```
sudo mysql -u root -p
```

```
...
```

```
```sql
```

```
CREATE DATABASE fraca_inventory;
```

```
CREATE USER 'inventory_user'@'localhost' IDENTIFIED BY  
'secure_password_here';
```

```
GRANT ALL PRIVILEGES ON fraca_inventory.* TO  
'inventory_user'@'localhost';
```

```
FLUSH PRIVILEGES;
```



```
EXIT;
```

```
```
```

```
```bash
```

```
# Run migrations and seeders
```

```
php artisan migrate --force
```

```
php artisan db:seed --force
```

```
```
```

```
Default seeded accounts:
```

```
| Email | Password | Role |
```

```
|-----|-----|-----|
```

```
| admin@inventory.com | password123 | Admin |
```

```
| manager@inventory.com | password123 | Manager |
```

```
| staff@inventory.com | password123 | Staff |
```

## 4.5 Web Server Configuration

### Nginx Configuration

```
```nginx
```

```
server {
```

```
    listen 80;
```

```
    server_name yourdomain.com;
```

```
    return 301 https://$server_name$request_uri;
```

```
}
```

```
server {
```

```
    listen 443 ssl http2;
```

```
    server_name yourdomain.com;
```

```
    root /var/www/fraca-inventory/public;
```

```
    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
```

```
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;
```

```
    add_header X-Frame-Options "SAMEORIGIN";
```

```
    add_header X-Content-Type-Options "nosniff";
```

```
    add_header X-XSS-Protection "1; mode=block";
```

```
    index index.php;
```

```
    location / {
```

```
        try_files $uri $uri/ /index.php?$query_string;
```

```
    }
```

```
    location ~ \.php$ {
```

```
        fastcgi_pass unix:/var/run/php/php8.2-fpm.sock;
```

```
        fastcgi_param SCRIPT_FILENAME
```

```
$realpath_root$fastcgi_script_name;
```

```
        include fastcgi_params;
```

```

    }

    location ~ /\.(!well-known).* {
        deny all;
    }
}
'''

```

4.6 Set Permissions

```

'''bash
# Set proper ownership
sudo chown -R www-data:www-data /var/www/fraca-inventory
sudo chmod -R 755 /var/www/fraca-inventory/storage
sudo chmod -R 755 /var/www/fraca-inventory/bootstrap/cache

# Restart services
sudo systemctl restart nginx
sudo systemctl restart php8.2-fpm
'''

```

4.7 Configure Scheduled Tasks

```

'''bash

```

```
# Add to crontab
```

```
crontab -e
```

```
'''
```

Add the following line to run Laravel scheduler every minute:

```
'''
```

```
* * * * * cd /var/www/fraca-inventory && php artisan schedule:run >>
/dev/null 2>&1
```

```
'''
```

****Scheduled tasks include:****

- `GenerateStockAlerts` command: Checks products below reorder level and creates alert records

4.8 Configure Queue Worker (if using database queue)

```
```bash
```

```
Create supervisor configuration
```

```
sudo nano /etc/supervisor/conf.d/laravel-worker.conf
```

```
'''
```

```
```ini
```

```
[program:laravel-worker]
```

```
process_name=%(program_name)s_%(process_num)02d
```

```
command=php /var/www/fraca-inventory/artisan queue:work --sleep=3 --  
tries=3 --max-time=3600
```

```
autostart=true
```

```
autorestart=true
```

```
stopasgroup=true
```

```
killasgroup=true
```

```
user=www-data
```

```
numprocs=2
```

```
redirect_stderr=true
```

```
stdout_logfile=/var/www/fraca-inventory/storage/logs/worker.log
```

```
stopwaitsecs=3600
```

```
``
```

```
```bash
```

```
Start supervisor
```

```
sudo supervisorctl reread
```

```
sudo supervisorctl update
```

```
sudo supervisorctl start laravel-worker:*
```

```
``
```

## 5. Production Optimizations

### 5.1 Caching

```
```bash  
php artisan config:cache  
php artisan route:cache  
php artisan view:cache  
php artisan event:cache  
```
```

### 5.2 Security Hardening

| Task                         | Command/Action                         |
|------------------------------|----------------------------------------|
| Set secure file permissions  | `chmod -R 755 storage bootstrap/cache` |
| Disable .env viewing         | Configure web server to deny access    |
| Enable HTTPS                 | SSL certificate configured             |
| Set secure session cookies   | `SESSION_SECURE_COOKIE=true`           |
| Set secure session cookies   | Set secure session cookies             |
| `SESSION_SECURE_COOKIE=true` |                                        |
| Restrict config caching      | `php artisan config:cache`             |
| Set proper .env permissions  | `chmod 640 .env`                       |

## 5.3 Monitoring Setup

```
```bash
# Install Laravel Telescope (optional, development only)
composer require laravel/telescope --dev
php artisan telescope:install
```
```

Log locations:

- Application logs: `storage/logs/laravel.log`
- Web server logs: `/var/log/nginx/access.log` and `/var/log/nginx/error.log`
- Worker logs: `storage/logs/worker.log`

## 6. Post-Deployment Verification

### 6.1 Verification Checklist

- [ ] Application loads at <https://yourdomain.com>
- [ ] Login works for `admin@inventory.com` / `password123`
- [ ] Login works for `manager@inventory.com` / `password123`
- [ ] Login works for `staff@inventory.com` / `password123`
- [ ] Dashboard displays statistics and widgets
- [ ] Products module: create, edit, view, delete (admin)
- [ ] Products module: delete triggers admin verification modal (manager)
- [ ] Sales module: create sale with dynamic total calculation
- [ ] Purchases module: create purchase with stock increment
- [ ] Reports: generate sales, purchases, stock levels, inventory valuation
- [ ] CSV export downloads correctly from all report tabs
- [ ] Low stock alerts appear on dashboard
- [ ] User management accessible only to Admin
- [ ] No JavaScript console errors
- [ ] Scheduled tasks running (check logs for `GenerateStockAlerts`)
- [ ] Database backups configured and tested



## 6.2 Database Connection Test

```
```bash
php artisan tinker
>>> DB::connection()->getPdo();
>>> DB::table('users')->count();
```
```

## 6.3 Schedule Test

```
```bash
php artisan schedule:list
php artisan schedule:run
```
```

---

## 7. Rollback Procedure

If deployment issues occur:

```
```bash
#Restore from previous version
cd /var/www
mv fraca-inventory fraca-inventory-failed
git clone https://your-repository-url/fraca-inventory.git fraca-inventory
cd fraca-inventory

#Copy previous environment and storage
cp ../fraca-inventory-failed/.env ./
cp -r ../fraca-inventory-failed/storage/* storage/

#Restore database from backup
mysql -u inventory_user -p fraca_inventory < /path/to/backup.sql

#Rebuild
composer install --optimize-autoloader --no-dev
npm install
npm run build
php artisan migrate --force
php artisan config:cache
```

```
php artisan route:cache
```

```
php artisan view:cache
```

```
#Restart services
```

```
sudo systemctl restart nginx
```

```
sudo systemctl restart php8.2-fpm
```

```
````
```

## 8. **Maintenance Mode**

To perform maintenance:

```
``bash
```

```
Enable maintenance mode
```

```
php artisan down --retry=60 --message="System undergoing maintenance. Please
try again in 1 minute."
```

```
Disable maintenance mode
```

```
php artisan up
```

```
````
```

9. Backup Strategy

9.1 Database Backup

```
```bash
```

```
Create backup script
```

```
sudo nano /etc/cron.daily/inventory-backup
```

```
```
```

```
```bash
```

```
#!/bin/bash
```

```
BACKUP_DIR="/var/backups/fraca-inventory"
```

```
DATE=$(date +%Y%m%d_%H%M%S)
```

```
mysqldump -u inventory_user -p'secure_password_here' fraca_inventory >
$BACKUP_DIR/db_$DATE.sql
```

```
Keep only last 30 days
```

```
find $BACKUP_DIR -name "db_*.sql" -mtime +30 -delete
```

```
```
```

```
```bash
```

```
Make executable
```

```
sudo chmod +x /etc/cron.daily/inventory-backup
```

```
```
```

9.2 File Backup

Backup `storage/` directory and `.env` file regularly.

10. Troubleshooting

Common Issues

| Issue | Solution |
|-----------------------------|--|
| Solution | |
| White screen (500 error) | Check `storage/logs/laravel.log` for details |
| 403 Forbidden | Check `storage/logs/laravel.log` for details |
| Database connection refused | Verify credentials in .env and MySQL user privileges |
| CSRF token mismatch | Ensure `SESSION_DOMAIN` matches your domain |
| Assets not loading | Database connection refused |
| Scheduler not running | Confirm crontab entry and `php artisan schedule:run` works |

11. Contact & Support

For deployment issues or questions, contact the development team with relevant log files from `storage/logs/laravel.log`.